Experiment 4

Objective: To write a program to remove left recursion from a grammar.

Procedure:

1. Create the C++ source file left_recursion.cpp.
2. In left_recursion.cpp, read the number of productions and then each production in the form A->alpha|beta.
3. For every production, split its right-hand side into alternatives. Alternatives beginning with the left-hand side are the left-recursive part (alpha); the remaining alternatives are the beta part.
4. If a production has no left-recursive alternative, print it unchanged. Otherwise rewrite it as A -> beta1A' | beta2A' | ... and A' -> alpha1A' | alpha2A' | ... | e, where e stands for epsilon.
5. Open the terminal in the directory containing left_recursion.cpp and run the following commands:

```
$ g++ left_recursion.cpp -o left_recursion
$ ./left_recursion
```

6. Enter the productions at the prompt and observe the grammar without left recursion.

Source Code:

```
#include <stdio.h>
#include <string.h>

#define MAXR 32
#define MAXP 32
#define MAXS 24
#define LEN  16

typedef struct { char sym[MAXS][LEN]; int n; } Prod;
typedef struct { char lhs[LEN]; Prod p[MAXP]; int np; } Rule;

Rule g[MAXR];
int nr = 0;

int find_rule(const char *lhs) {
    for (int i = 0; i < nr; i++)
        if (strcmp(g[i].lhs, lhs) == 0) return i;
    return -1;
}

int insert_rule(const char *lhs, int pos) {
    for (int i = nr; i > pos; i--) g[i] = g[i - 1];
    strcpy(g[pos].lhs, lhs);
    g[pos].np = 0;
    nr++;
    return pos;
}

void read_grammar(void) {
    char line[512];
    printf("Enter the grammar (one rule per line, blank line to finish):\n");
    while (fgets(line, sizeof(line), stdin)) {
        char *arrow = strstr(line, "->");
        if (arrow == NULL) {
            if (nr > 0) break;
            continue;
        }
```

```
            char lhs[LEN] = "";
            for (char *s = line; s < arrow; s++)
                if (*s != ' ' && *s != '\t') strncat(lhs, s, 1);
            int idx = find_rule(lhs);
            if (idx == -1) idx = insert_rule(lhs, nr);

            char *save1, *save2;
            char *part = strtok_r(arrow + 2, "|", &save1);
            while (part != NULL) {
                Prod pr; pr.n = 0;
                char *tok = strtok_r(part, " \t\r\n", &save2);
                int is_eps = (tok != NULL && strcmp(tok, "eps") == 0);
                while (tok != NULL && !is_eps) {
                    strcpy(pr.sym[pr.n++], tok);
                    tok = strtok_r(NULL, " \t\r\n", &save2);
                }
                g[idx].p[g[idx].np++] = pr;
                part = strtok_r(NULL, "|", &save1);
            }
        }
    }

    void remove_direct(int idx) {
        Prod rec[MAXP], non[MAXP];
        int nrec = 0, nnon = 0;
        for (int i = 0; i < g[idx].np; i++) {
            Prod *p = &g[idx].p[i];
            if (p->n > 0 && strcmp(p->sym[0], g[idx].lhs) == 0) {
                Prod a; a.n = p->n - 1;
                for (int k = 0; k < a.n; k++) strcpy(a.sym[k], p->sym[k + 1]);
                rec[nrec++] = a;
            } else non[nnon++] = *p;
        }



        if (nrec == 0) return;

        char name[LEN + 8];
        snprintf(name, sizeof(name), "%s'", g[idx].lhs);
        while (find_rule(name) != -1) strcat(name, "'");

        int nidx = insert_rule(name, idx + 1);
        g[idx].np = 0;
        for (int i = 0; i < nnon; i++) {
            Prod np = non[i];
            strcpy(np.sym[np.n++], name);
            g[idx].p[g[idx].np++] = np;
        }
        for (int i = 0; i < nrec; i++) {
            Prod np = rec[i];
            strcpy(np.sym[np.n++], name);
            g[nidx].p[g[nidx].np++] = np;
        }
        Prod eps; eps.n = 0;
        g[nidx].p[g[nidx].np++] = eps;
    }

    void remove_left_recursion(void) {
        char orig[MAXR][LEN];
        int on = nr;
        for (int i = 0; i < on; i++) strcpy(orig[i], g[i].lhs);
        for (int i = 0; i < on; i++) {
```

```
        for (int j = 0; j < i; j++) {
            int ai = find_rule(orig[i]), aj = find_rule(orig[j]);
            Prod out[MAXP]; int nout = 0;
            for (int k = 0; k < g[ai].np; k++) {
                Prod *p = &g[ai].p[k];
                if (p->n > 0 && strcmp(p->sym[0], orig[j]) == 0) {
                    for (int d = 0; d < g[aj].np; d++) {
                        Prod np = g[aj].p[d];
                        for (int s = 1; s < p->n; s++)
                            strcpy(np.sym[np.n++], p->sym[s]);
                        out[nout++] = np;
                    }
                } else out[nout++] = *p;
            }
            g[ai].np = nout;
            for (int k = 0; k < nout; k++) g[ai].p[k] = out[k];
        }
        remove_direct(find_rule(orig[i]));
    }
}

void print_grammar(void) {
    for (int i = 0; i < nr; i++) {
        printf("%s -> ", g[i].lhs);
        for (int k = 0; k < g[i].np; k++) {
            Prod *p = &g[i].p[k];
            if (k > 0) printf(" | ");
            if (p->n == 0) printf("eps");
            for (int s = 0; s < p->n; s++)
                printf("%s%s", s ? " " : "", p->sym[s]);
        }
        printf("\n");
    }
}

int main(void) {
    read_grammar();
    remove_left_recursion();
    printf("\nGrammar after removing left recursion:\n");
    print_grammar();
    return 0;
}
```

Results:

```
Enter the number of productions: 3
Enter the productions (e.g. E->E+T|T):
E->E+T|T
T->T*F|F
F->(E)|id
E->TE'
E'->+TE'|e
T->FT'
T'->*FT'|e
F->(E)|id
```

Learning Outcome:

- Understood left recursion and why it causes infinite recursion in a top-down parser.
- Learnt the standard transformation: A -> A alpha | beta becomes A -> beta A' and A' -> alpha A' | epsilon.
- Saw that the transformed grammar generates the same language without left recursion.

Experiment 5

Objective: To write a program to perform left factoring on a grammar.

Procedure:

1. Create the C++ source file left_factoring.cpp.
2. In left_factoring.cpp, read a production in the form A->alpha beta1 | alpha beta2 | ... .
3. Find the longest common prefix of the alternatives that share a prefix with the first alternative.
4. Rewrite the production as A -> prefix A' | (other alternatives) and A' -> suffix1 | suffix2 | ..., using e for epsilon when a suffix is empty.
5. If no two alternatives share a common prefix, report that no left factoring is required.
6. Open the terminal in the directory containing left_factoring.cpp and run the following commands:

```
$ g++ left_factoring.cpp -o left_factoring
$ ./left_factoring
```

7. Enter the production at the prompt and observe the factored grammar.

Source Code:

left_factoring.cpp

```
#include <bits/stdc++.h>
using namespace std;

string commonPrefix(string a, string b) {
    string prefix = "";
    for (int i = 0; i < (int)min(a.size(), b.size()); i++) {
        if (a[i] == b[i]) prefix += a[i];
        else break;
    }
    return prefix;
}

int main() {
    string prod;
    cout << "Enter the production (e.g. S->iEtS|iEtSeS|a): ";
    cin >> prod;
    string lhs = prod.substr(0, prod.find("->"));
    string rhs = prod.substr(prod.find("->") + 2);

    vector<string> alternatives;
    string temp = "";
    for (char c : rhs) {
        if (c == '|') { alternatives.push_back(temp); temp = ""; }
        else temp += c;
    }
    alternatives.push_back(temp);

    // Group alternatives sharing a common prefix with the first one
    string prefix = "";
    vector<string> group, rest;
    for (int i = 1; i < (int)alternatives.size(); i++) {
        string p = commonPrefix(alternatives[0], alternatives[i]);
        if (p.size() > prefix.size()) prefix = p;
    }
    group.push_back(alternatives[0]);
    for (int i = 1; i < (int)alternatives.size(); i++) {
        if (alternatives[i].rfind(prefix, 0) == 0 && !prefix.empty())
```

```
            group.push_back(alternatives[i]);
        else
            rest.push_back(alternatives[i]);
    }

    if (prefix.empty() || group.size() < 2) {
        cout << "No left factoring required." << endl;
        cout << prod << endl;
    } else {
        string prime = lhs + "'";
        cout << lhs << "->" << prefix << prime;
        for (string r : rest) cout << "|" << r;
        cout << endl;
        cout << prime << "->";
        for (int i = 0; i < (int)group.size(); i++) {
            string suffix = group[i].substr(prefix.size());
            if (suffix.empty()) suffix = "e";
            if (i > 0) cout << "|";
            cout << suffix;
        }
        cout << endl;
    }
    return 0;
}
```

Results:

```
Enter the production (e.g. S->iEtS|iEtSeS|a): S->iEtS|iEtSeS|a
S->iEtSS'|a
S'->e|eS
```

Learning Outcome:

- Understood the common prefix problem and why it makes predictive parsing fail.
- Learnt the left factoring transformation: A -> alpha beta1 | alpha beta2 becomes A -> alpha A' and A' -> beta1 | beta2.
- Saw that an empty suffix after factoring becomes epsilon, as in S' -> e | eS.
- Understood that left factoring postpones the decision until enough input has been seen.

Experiment 6

Objective: To write a program to show all the operations of a stack.

Procedure:

1. Create the C++ source file stack.cpp.
2. In stack.cpp, implement a stack of size 5 using an array and a variable top, initialised to -1.
3. Implement push (insert at top, reporting Stack Overflow when the stack is full), pop (remove from top, reporting Stack Underflow when the stack is empty), peek (display the top element) and display (print all elements from top to bottom).
4. In main(), display a menu and repeatedly read the user's choice until the exit option is chosen.
5. Open the terminal in the directory containing stack.cpp and run the following commands:

```
$ g++ stack.cpp -o stack
$ ./stack
```

6. Perform push, display, peek and pop operations from the menu and observe the output.

Source Code:

stack.cpp

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int stack[5], top = -1, choice, value;
    cout << "Stack Operations: 1.Push 2.Pop 3.Peek 4.Display 5.Exit" << endl;
    while (true) {
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice) {
            case 1:
                if (top == 4) cout << "Stack Overflow" << endl;
                else {
                    cout << "Enter the value to push: ";
                    cin >> value;
                    stack[++top] = value;
                    cout << value << " pushed onto the stack" << endl;
                }
                break;
            case 2:
                if (top == -1) cout << "Stack Underflow" << endl;
                else cout << stack[top--] << " popped from the stack" << endl;
                break;
            case 3:
                if (top == -1) cout << "Stack is empty" << endl;
                else cout << "Top element is " << stack[top] << endl;
                break;
            case 4:
                if (top == -1) cout << "Stack is empty" << endl;
                else {
                    cout << "Stack elements: ";
                    for (int i = top; i >= 0; i--) cout << stack[i] << " ";
                    cout << endl;
                }
                break;
            case 5:
                return 0;
            default:
                cout << "Invalid choice" << endl;
        }}}
```

Results:

```
Stack Operations: 1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter your choice: 1
Enter the value to push: 10
10 pushed onto the stack
Enter your choice: 1
Enter the value to push: 20
20 pushed onto the stack
Enter your choice: 1
Enter the value to push: 30
30 pushed onto the stack
Enter your choice: 4
Stack elements: 30 20 10
Enter your choice: 3
Top element is 30
Enter your choice: 2
30 popped from the stack
Enter your choice: 4
Stack elements: 20 10
Enter your choice: 5
```

Learning Outcome:

- Understood the stack data structure and its LIFO (Last In First Out) order.
- Implemented push, pop, peek and display operations using an array and a top pointer.
- Learned to detect the boundary conditions: Stack Overflow on a full stack and Stack Underflow on an empty stack.
- Understood why the stack is central to parsing: shift-reduce parsers and recursive descent both use it to hold grammar symbols.

Experiment 7

Objective: To write a program to find out the LEADING of the non-terminals in a grammar.

Procedure:

1. Create the C++ source file leading.cpp.
2. In leading.cpp, read the productions of the grammar and split every right-hand side at '|' into separate alternatives.
3. Apply the rules for LEADING repeatedly until no set changes: if a production is A -> a alpha, where a is a terminal, add a to LEADING(A); if it is A -> B alpha, where B is a non-terminal, add all of LEADING(B) to LEADING(A), and if alpha begins with a terminal, add that terminal to LEADING(A) as well.
4. Open the terminal in the directory containing leading.cpp and run the following commands:

```
$ g++ leading.cpp -o leading
$ ./leading
```

5. Enter the productions at the prompt and observe the LEADING sets.

Source Code:

leading.cpp

```
#include <bits/stdc++.h>
using namespace std;

bool isNonTerminal(char c) { return c >= 'A' && c <= 'Z'; }

int main() {
    int n;
    cout << "Enter the number of productions: ";
    cin >> n;
    cout << "Enter the productions (e.g. E->E+T|T):" << endl;
    vector<pair<char, string>> prods;
    vector<char> nts;
    for (int i = 0; i < n; i++) {
        string p; cin >> p;
        char lhs = p[0];
        string rhs = p.substr(3);
        if (find(nts.begin(), nts.end(), lhs) == nts.end())
           nts.push_back(lhs);
        string alt = "";
        for (int j = 0; j <= (int)rhs.size(); j++) {
            if (j == (int)rhs.size() || rhs[j] == '|') {
                prods.push_back({lhs, alt});
                alt = "";
            } else alt += rhs[j];
        }
    }

    map<char, set<char>> leading;
    bool changed = true;
    while (changed) {
        changed = false;
        for (auto [A, rhs] : prods) {
            int before = leading[A].size();
            char X1 = rhs[0];
            if (!isNonTerminal(X1)) {
```

```
                leading[A].insert(X1);
            } else {
                for (char t : leading[X1]) leading[A].insert(t);
                if (rhs.size() > 1 && !isNonTerminal(rhs[1]))
                    leading[A].insert(rhs[1]);
            }
            if ((int)leading[A].size() > before) changed = true;
        }
    }

    for (char A : nts) {
        cout << "LEADING(" << A << ") = { ";
        for (char t : leading[A]) cout << t << " ";
        cout << "}" << endl;
    }
    return 0;
}
```

Results:

```
Enter the number of productions: 3
Enter the productions (e.g. E->E+T|T):
E->E+T|E-T|T
T->T*F|T/F|F
F->(E)|i
LEADING(E) = { ( * + - / i }
LEADING(T) = { ( * / i }
LEADING(F) = { ( i }
```

Learning Outcome:

- Understood the definition of LEADING: the set of terminals that can appear as the first terminal in any string derived from a non-terminal.
- Learn the fixed-point computation: sets are updated repeatedly until no new terminal is added.
- Saw how operators enter LEADING sets through productions such as E -> E+T, which puts + in LEADING(E).
- Understood that LEADING sets are used to build the operator precedence table.

Experiment 8

Objective: To write a program to implement shift-reduce parsing for a string.

Procedure:

1. Create the C++ source file shift_reduce.cpp.
2. In shift_reduce.cpp, use the grammar E -> E+E | E*E | (E) | i and the input string i+i*i$.
3. Maintain a stack (initially $) and the remaining input. Repeatedly shift the next input symbol onto the stack, or reduce a handle on top of the stack: i to E, (E) to E, E+E to E and E*E to E.
4. Print the stack, the remaining input and the action at every step, and accept when the stack is $E and the input is $.
5. Open the terminal in the directory containing shift_reduce.cpp and run the following commands:

```
$ g++ shift_reduce.cpp -o shift_reduce
$ ./shift_reduce
```

6. Observe the shift-reduce parsing table and verify that the string is accepted.

Source Code:

shift_reduce.cpp

```
#include <bits/stdc++.h>
using namespace std;

bool endsWith(string s, string suffix) {
    return s.size() >= suffix.size() &&
           s.compare(s.size() - suffix.size(), suffix.size(), suffix)
== 0;
}

int main() {
    // Grammar: E -> E+E | E*E | (E) | i
    string input = "i+i*i$";
    string stack = "$";
    int pos = 0;
    cout << "Stack\tInput\tAction" << endl;
    while (true) {
        string curStack = stack;
        string curInput = input.substr(pos);
        if (stack == "$E" && curInput == "$") {
            cout << curStack << "\t" << curInput << "\tAccept" << endl;
            break;
        }
        string action;
        if (endsWith(stack, "(E)")) {
            stack = stack.substr(0, stack.size() - 3) + "E";
            action = "Reduce E->(E)";
        } else if (endsWith(stack, "E+E") && (curInput[0] == '+' ||
curInput[0] == '$')) {
            stack = stack.substr(0, stack.size() - 3) + "E";
            action = "Reduce E->E+E";
        } else if (endsWith(stack, "E*E") && (curInput[0] == '*' ||
curInput[0] == '$')) {
            stack = stack.substr(0, stack.size() - 3) + "E";
            action = "Reduce E->E*E";
        } else if (endsWith(stack, "i")) {
            stack = stack.substr(0, stack.size() - 1) + "E";
            action = "Reduce E->i";
```

```
        } else {
            action = "Shift";
            stack += input[pos];
            pos++;
        }
        cout << curStack << "\t" << curInput << "\t" << action << endl;
    }
    return 0;
}
```

Results:

```
Stack    Input    Action
$        i+i*i$   Shift
$i       +i*i$    Reduce E->i
$E       +i*i$    Shift
$E+      i*i$     Shift
$E+i     *i$      Reduce E->i
$E+E     *i$      Shift
$E+E*    i$       Shift
$E+E*i   $        Reduce E->i
$E+E*E   $        Reduce E->E*E
$E+E     $        Reduce E->E+E
$E       $        Accept
```

Learning Outcome:

- Understood shift-reduce parsing: shifting input symbols onto the stack and reducing handles to non-terminals.
- Learn the four actions of a shift-reduce parser: shift, reduce, accept and error.
- Saw how the parser delays reducing E+E while * follows, which respects the higher precedence of *.
- Verified that the string i+i*i is accepted, since the stack finally becomes $E with no input left.

Experiment 9

Objective: To write a program to find out the FIRST of the non-terminals in a grammar.

Procedure:

1. Create the C++ source file first.cpp.
2. In first.cpp, read the productions of the grammar and split every right-hand side at '|' into separate alternatives.
3. Apply the rules for FIRST repeatedly until no set changes: for a production A -> X1 X2 ... Xn, add FIRST(X1) except epsilon to FIRST(A); if epsilon is in FIRST(X1), continue with X2, and so on; if every Xi can derive epsilon, add epsilon to FIRST(A). A terminal contributes itself.
4. Use the grammar E -> TR, R -> +TR | e, T -> FU, U -> *FU | e, F -> (E) | i, where R stands for E', U stands for T' and e stands for epsilon.
5. Open the terminal in the directory containing first.cpp and run the following commands:

```
$ g++ first.cpp -o first
$ ./first
```

6. Enter the productions at the prompt and observe the FIRST sets.

Source Code:

first.cpp

```
#include <bits/stdc++.h>
using namespace std;

bool isNonTerminal(char c) { return c >= 'A' && c <= 'Z'; }

int main() {
    // Grammar: E->TR, R->+TR|e, T->FU, U->*FU|e, F->(E)|i
    // (R stands for E', U stands for T', e stands for epsilon)
    int n;
    cout << "Enter the number of productions: ";
    cin >> n;
    cout << "Enter the productions (e stands for epsilon):" << endl;
    vector<pair<char, string>> prods;
    vector<char> nts;
    for (int i = 0; i < n; i++) {
        string p; cin >> p;
        char lhs = p[0];
        string rhs = p.substr(3);
        if (find(nts.begin(), nts.end(), lhs) == nts.end())
           nts.push_back(lhs);
        string alt = "";
        for (int j = 0; j <= (int)rhs.size(); j++) {
            if (j == (int)rhs.size() || rhs[j] == '|') {
                prods.push_back({lhs, alt});
                alt = "";
            } else alt += rhs[j];
        }
    }

    map<char, set<char>> first;
    bool changed = true;
    while (changed) {
        changed = false;
        for (auto [A, rhs] : prods) {
```

```
            int before = first[A].size();
            bool allEpsilon = true;
            for (char X : rhs) {
                if (!isNonTerminal(X)) {
                    first[A].insert(X);
                    allEpsilon = false;
                    break;
                } else {
                    for (char t : first[X])
                        if (t != 'e') first[A].insert(t);
                    if (first[X].count('e') == 0) {
                        allEpsilon = false;
                        break;
                    }
                }
            }
            if (allEpsilon) first[A].insert('e');
            if ((int)first[A].size() > before) changed = true;
        }
    }

    for (char A : nts) {
        cout << "FIRST(" << A << ") = { ";
        for (char t : first[A]) {
            if (t == 'e') cout << "epsilon ";
            else cout << t << " ";
        }
        cout << "}" << endl;
    }
    return 0;
}
```

Results:

```
Enter the number of productions: 5
Enter the productions (e stands for epsilon):
E->TR
R->+TR|e
T->FU
U->*FU|e
F->(E)|i
FIRST(E) = { ( i }
FIRST(R) = { + epsilon }
FIRST(T) = { ( i }
FIRST(U) = { * epsilon }
FIRST(F) = { ( i }
```

Learning Outcome:

- Understood the definition of FIRST: the set of terminals that can begin a string derived from a non-terminal.
- Learn how epsilon propagates through a production: if X1 derives epsilon, FIRST(X2) is also included.
- Saw why FIRST(R) contains epsilon: R -> e is a production of the grammar.
- Understood that FIRST sets are used to build the LL(1) predictive parsing table.

Experiment 10

Objective: To write a program to check whether a grammar is operator precedent (operator precedence).

Procedure:

1. Create the C++ source file operator_precedence.cpp.
2. In operator_precedence.cpp, first check that the grammar is an operator grammar: no production may derive epsilon, and no right-hand side may contain two adjacent non-terminals.
3. Compute the LEADING and TRAILING sets of every non-terminal: LEADING as in Experiment 7, and TRAILING by the mirror-image rules (a terminal at the end of a production, the TRAILING set of a final non-terminal, and a terminal immediately before a final non-terminal).
4. Build the precedence relations from every pair of adjacent symbols on a right-hand side: a terminal followed by a non-terminal gives yields-precedence (<) with the non-terminal's LEADING set, a non-terminal followed by a terminal gives takes-precedence (>) with the non-terminal's TRAILING set, and two adjacent terminals give equal precedence (=).
5. If any pair of terminals gets more than one relation, report a conflict: the grammar is not an operator precedence grammar. Otherwise it is.
6. Open the terminal in the directory containing operator_precedence.cpp and run the following commands:

```
$ g++ operator_precedence.cpp -o operator_precedence
$ ./operator_precedence
```

7. Enter the productions at the prompt and observe the table and the verdict.

Source Code:

operator_precedence.cpp

```
#include <bits/stdc++.h>
using namespace std;

bool isNonTerminal(char c) { return c >= 'A' && c <= 'Z'; }

int main() {
    int n;
    cout << "Enter the number of productions: ";
    cin >> n;
    cout << "Enter the productions:" << endl;
    vector<pair<char, string>> prods;
    vector<char> nts;
    vector<char> terms;
    for (int i = 0; i < n; i++) {
        string p; cin >> p;
        char lhs = p[0];
        string rhs = p.substr(3);
        if (find(nts.begin(), nts.end(), lhs) == nts.end())
           nts.push_back(lhs);
        string alt = "";
        for (int j = 0; j <= (int)rhs.size(); j++) {
            if (j == (int)rhs.size() || rhs[j] == '|') {
                prods.push_back({lhs, alt});
                for (char c : alt)
                    if (!isNonTerminal(c) && find(terms.begin(),
terms.end(), c) == terms.end())
                        terms.push_back(c);
                alt = "";
            } else alt += rhs[j];
```

```
        }
    }

    // Operator grammar checks: no epsilon productions, no adjacent
non-terminals
    bool operatorGrammar = true;
    for (auto [A, rhs] : prods) {
        if (rhs == "e") operatorGrammar = false;
        for (int i = 0; i + 1 < (int)rhs.size(); i++)
            if (isNonTerminal(rhs[i]) && isNonTerminal(rhs[i + 1]))
                operatorGrammar = false;
    }
    if (!operatorGrammar) {
        cout << "The grammar is NOT an operator grammar." << endl;
        cout << "So, the grammar is NOT an operator precedence
grammar." << endl;
        return 0;
    }

    // LEADING and TRAILING
    map<char, set<char>> leading, trailing;
    bool changed = true;
    while (changed) {
        changed = false;
        for (auto [A, rhs] : prods) {
            int b1 = leading[A].size(), b2 = trailing[A].size();
            char X1 = rhs[0], Xn = rhs[rhs.size() - 1];
            if (!isNonTerminal(X1)) leading[A].insert(X1);
            else {
                for (char t : leading[X1]) leading[A].insert(t);
                if (rhs.size() > 1 && !isNonTerminal(rhs[1]))
leading[A].insert(rhs[1]);
            }
            if (!isNonTerminal(Xn)) trailing[A].insert(Xn);
            else {
                for (char t : trailing[Xn]) trailing[A].insert(t);
                if (rhs.size() > 1 && !isNonTerminal(rhs[rhs.size() -
2]))
                    trailing[A].insert(rhs[rhs.size() - 2]);
            }
            if ((int)leading[A].size() > b1 || (int)trailing[A].size()
> b2) changed = true;
        }
    }

    for (char A : nts) {
        cout << "LEADING(" << A << ") = { ";
        for (char t : leading[A]) cout << t << " ";
        cout << "}" << endl;
    }
    for (char A : nts) {
        cout << "TRAILING(" << A << ") = { ";
        for (char t : trailing[A]) cout << t << " ";
        cout << "}" << endl;
    }
```

```
    // Precedence relations: '<' = yields precedence, '=' = equal, '>'
= takes precedence
    map<pair<char, char>, set<char>> rel;
    for (auto [A, rhs] : prods) {
        for (int i = 0; i + 1 < (int)rhs.size(); i++) {
            char X = rhs[i], Y = rhs[i + 1];
            if (!isNonTerminal(X) && !isNonTerminal(Y)) {
                rel[{X, Y}].insert('=');
            } else if (!isNonTerminal(X) && isNonTerminal(Y)) {
                for (char t : leading[Y]) rel[{X, t}].insert('<');
            } else if (isNonTerminal(X) && !isNonTerminal(Y)) {
                for (char t : trailing[X]) rel[{t, Y}].insert('>');
            }
        }
    }

    cout << "Operator Precedence Table:" << endl;
    cout << "     ";
    for (char b : terms) cout << b << "   ";
    cout << endl;
    for (char a : terms) {
        cout << a << "    ";
        for (char b : terms) {
            if (rel.count({a, b}) && rel[{a, b}].size() == 1)
                cout << *rel[{a, b}].begin() << "   ";
            else
                cout << "    ";
        }
        cout << endl;
    }

    bool conflict = false;
    for (auto &entry : rel) {
        if (entry.second.size() > 1) {
            cout << "Conflict between " << entry.first.first << " and "
                 << entry.first.second << endl;
            conflict = true;
        }
    }
    if (conflict)
        cout << "The grammar is NOT an operator precedence grammar." <<
endl;
    else
        cout << "The grammar IS an operator precedence grammar." <<
endl;
    return 0;
}
```

Results:

```
Enter the number of productions: 3
Enter the productions:
E->E+T|T
T->T*F|F
F->(E)|i
LEADING(E) = { ( * + i }
LEADING(T) = { ( * i }
```

```
LEADING(F) = { ( i }
TRAILING(E) = { ) * + i }
TRAILING(T) = { ) * i }
TRAILING(F) = { ) i }
Operator Precedence Table:
     +   *   (   )   i
+    >   <   <   >   <
*    >   >   <   >   <
(    <   <   <       <
)    >   >       >
i    >   >       >
The grammar IS an operator precedence grammar.

Enter the number of productions: 1
Enter the productions:
E->E+E|E*E|(E)|i
LEADING(E) = { ( * + i }
TRAILING(E) = { ) * + i }
Operator Precedence Table:
     +   *   (   )   i
+            <   >   <
*            <   >   <
(    <   <   <       <
)    >   >       >
i    >   >       >
Conflict between * and *
Conflict between * and +
Conflict between + and *
Conflict between + and +
The grammar is NOT an operator precedence grammar.
```

Learning Outcome:

- Understood the operator grammar conditions: no epsilon productions and no two adjacent non-terminals.
- Learned how LEADING and TRAILING sets generate the <, = and > precedence relations.
- Saw that the grammar E -> E+T | T, T -> T*F | F, F -> (E) | i has a conflict-free table and is an operator precedence grammar.
- Saw why E -> E+E | E*E | (E) | i fails: pairs such as (+, +) get both < and >, which is a conflict caused by the grammar's ambiguity.